

Monte Carlo Methods for Model Free Prediction and Control

Imagine You're Playing Super Mario Bros

You want to find the best strategy to finish a level - jumping over pits, avoiding enemies, and collecting coins.

1. Model-based (Planning)

2. Model-free (Learning by Playing)



Model-based (Planning)

- Suppose you **know everything** about how the game works:
 - You know exactly how Mario moves when you press each button.
 - You know where every enemy will appear and how fast they move.
 - You know how jumping at a specific pixel will land you safely on a platform.
- With all this knowledge, you could write down the game's rules as equations and calculate, step by step,
 - 👉 *"If I press jump now, I'll land there."*
 - 👉 *"If I move right for 3 steps, I'll reach the coin."*
- Then you could compute the best sequence of actions mathematically before even playing.
- That's **model-based planning** — like having the entire physics and map of the game coded into your brain.

Model-free (Learning by Playing)

- Now imagine you don't know how the game works exactly.
 - You **just start playing**:
 - Sometimes you jump too early and fall. Sometimes you wait too long and get hit by an enemy.
 - Gradually, you learn what works and what doesn't — by trying.
- You might notice:
 - *“Jump right after the enemy moves — that works.”*
 - *“Don't run too fast on that bridge — it collapses.”*
- You're not calculating probabilities or modeling the game's physics —
- you're collecting experiences and using them to improve your policy (how you act).
- That's **model-free learning** — no equations, just experience.

Why Use Model-free?

- In many real-world problems — like games, robots, or finance — **it's really hard to write exact equations for how things behave.**
- But it's easy to simulate or experience the world:
 - *Play Mario 1,000 times.*
 - *Let your robot try to walk 1,000 times.*
 - *Let your trading agent test on past data 1,000 times.*
- By doing this, the agent learns good actions without ever knowing the full rules.

Type	What You Know	How You Learn	Example
Model-based (Planning)	You know the full game rules and transitions	Use logic and equations to plan best actions	Knowing exact Mario physics and pre-calculating best moves
Model-free (Learning)	You don't know how actions change the world	Play and learn from trial and error	Learning Mario timing by playing again and again

The Big Idea

When we don't know how the world works exactly, we learn by trying things out and observing what happens — just like in real life.



No physics model needed — sample outcomes, average results, estimate truth

In Reinforcement Learning Terms

In Dynamic Programming (previous chapter),
we **knew the transition dynamics** — how one action leads to a new state.

But now, we **don't know** how the world behaves (model-free).

So, what do we do?

👉 We **sample experience**:

- Start from some state **S**
- Follow a policy (take actions)
- Record the total reward (called the **return**) until the episode ends
(like reaching the end of a game level)

Then, we repeat this **many times** and **average all returns** from that state.

That average = our **estimate of how good that state is** under that policy $\rightarrow v_{\pi}(s) \approx$
average return from state s

The Statistical Backbone

The Law of Large Numbers says:

If you take enough samples,

the average of those samples $\approx$ the true expected value.

So, if we play enough times,

our estimated value gets very close to the true value.

Why Monte Carlo (MC) Methods?

- Model dynamics $p(s', r \mid s, a)$ may be unknown
- Learn by sampling experience from environment/simulation
- Useful when sampling is easier than computing transitions (e.g., blackjack)
- MC is one major class of model-free learning (along with TD)



Monte Carlo in Action

- For large or complex environments, **we don't need to explore the entire world** —just the parts we actually visit while following our policy.
- That's what makes **MC efficient and scalable**.
- In fact, a variant called **Monte Carlo Tree Search (MCTS)** was famously used by **OpenAI** and **DeepMind** to train **AlphaGo** — the agent that beat world champions at the game Go.

Why It's Different from Dynamic Programming?

Concept	Dynamic Programming	Monte Carlo
Needs model?	✅ Yes (knows transition probabilities)	❌ No (just samples outcomes)
Works on	Any MDP	Only episodic MDPs (must end)
Uses Markov assumption?	✅ Yes (future depends only on current state)	❌ No (uses full return till episode end)
How it learns	Calculates expected values using model	Averages observed returns

Monte Carlo Prediction (State-Value Estimation)

- Goal: estimate $v_{\pi}(s) = \mathbb{E}[G_t \mid S_t = s]$
- Idea: replace expectation with average of sampled returns
- Run many episodes under policy π ; compute returns from visits to s
- Estimate improves with more samples (law of large numbers)



One visit is noisy; many visits estimate the true average.

When MC Works (and Why It's Different)

- Requires **episodic MDPs** (need termination to compute full return)
- **Does not work in non-terminating** environments
- MC uses full return; does not depend on **successor-state** values
- Described as **not** making a **Markov-property assumption** in the update

Episodic Task: Full Semester



Continuing Task: Never ends



MC needs episodes: the return is known only at termination.

First-Visit vs Every-Visit MC Prediction

- First-visit MC: update from first time state s appears in an episode
- Every-visit MC: update from every occurrence of s in the episode
- Both average returns G observed from s to episode end
- Every-visit is implemented by dropping the "first-visit check"

First-Visit Monte Carlo

- In this method, for each episode, you only consider the **first time** you reach that state.
- So, in one game:
 - You first reach “midway point” at time t_1 →  record total return from there till the end.
 - Later you might reach the same point again →  ignore those later visits in this episode.
- After many such episodes, you take the average of all these first-visit returns — that’s your estimate for $v_{\pi}(s)$.
- *“Only note down what happens the first time I reach this place in each game.”*

Every-Visit Monte Carlo

- In this method, for each episode, **you take all visits to that state.**
- So if you hit the same checkpoint three times in a single game,
- you record the **total return** from each of those visits till the end.
- Then, across **all episodes** and all visits, you average all those returns — that gives you $v_{\pi}(s)$.
- *“Every time I reach this place, I note what happens next.”*

Summary

Method

What You Record

Example

First-Visit MC

Only the *first time* you visit state s in an episode

First time Mario reaches checkpoint

Every-Visit MC

Every time you visit state s in an episode

Every time Mario revisits checkpoint

Goal

Estimate $v_{\pi}(s)$ = average total reward from that state until episode ends

state until episode ends

Restaurant = state, Day = episode

First-visit MC



Every-visit MC



First-visit: record first time; Every-visit: record every time.

In simple terms..

- You want to know **how good a situation (state) is**.
- You play many times and look at what happens from that point till the end.
 - **First-visit MC:** look only at the first time it happens per play.
 - **Every-visit MC:** look at every time it happens per play.
- Then, take the average of all those outcomes to estimate how good that state is.

Algorithm / Pseudo Code

Input:

π , the policy to be evaluated

Initialize:

Cumulative state values $S(s) = 0$ for all $s \in S$

Estimated state values $V(s) = 0$ for all $s \in S$

Visit count $N(s) = 0$ for all $s \in S$

Loop:

Sample episode following policy $\pi : S_0, A_0, R_1, S_1, A_1, R_2, \dots, A_{T-1}, R_T, S_T$

$G \leftarrow 0$

Loop backward for each step of episode, $t = T - 1, T - 2, \dots, 1, 0$

$G \leftarrow \gamma \cdot G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$

$N(s) \leftarrow N(s) + 1$

$S(s) \leftarrow S(s) + G$

Step	What Happens	Real-life Analogy
Initialize	Set everything to zero	Start with a clean notebook
Sample Episode	Play the game once using your current strategy	Play one full round of a game
Loop Backward	Calculate total reward from each step to end	Recall how much reward you got after each move
First-Visit Update	Record only first time you reach a place	Note only your <i>first</i> impression per game
Repeat Many Times	Keep averaging over episodes	The more you play, the better your estimate

Numerical

For each state $s \in \{A, B\}$:

- Cumulative sum $S(s) = 0$
- Visit count $N(s) = 0$
- Estimated value $V(s) = S(s)/N(s)$ (undefined until $N(s) > 0$, treat as 0 initially)

So initially:

- $S(A) = 0, N(A) = 0, V(A) = 0$
- $S(B) = 0, N(B) = 0, V(B) = 0$

Online (Incremental) MC Update Rules

- Maintain visit count $N(s)$ and estimate $V(s)$
- Incremental update (after observing return G):
 - $V(s) \leftarrow V(s) + \frac{1}{N(s)} (G - V(s))$
- Alternative: constant step-size α instead of $1/N$
- Constant α suggested for nonstationary problems

Concept**First-Visit MC****Every-Visit MC**

What we use

Only the **first time** a state appears in an episode**Every time** a state appears in an episode

Number of samples

Fewer (1 per state per episode)

More (multiple per state per episode)

Data used

Less

More

Implementation

Slightly more complex (must check for "first visit")

Simpler (no need to check)

In Simple Words

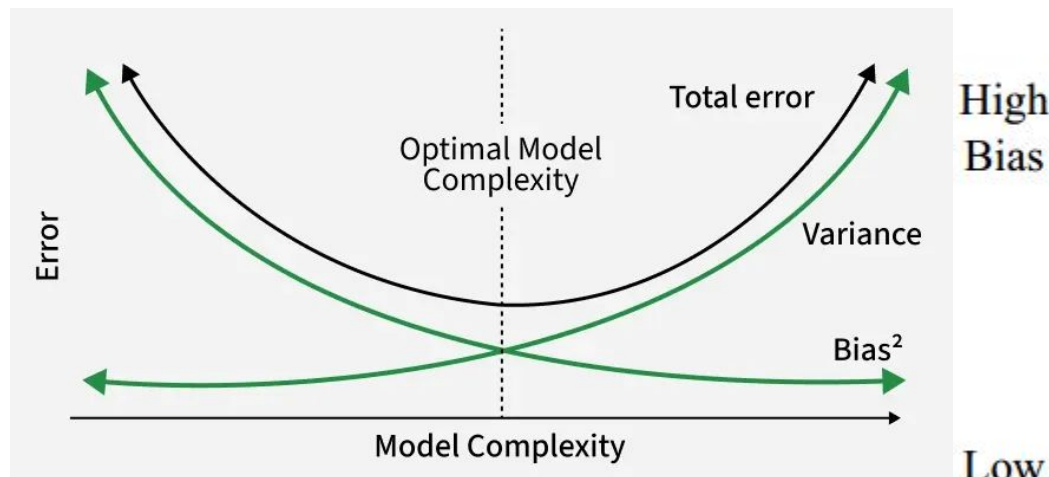
 First-Visit MC = *“Learn once per episode per state.”*

It's neat, clean, and less correlated.

 Every-Visit MC = *“Learn every time you see the state.”*

It's practical, fast, and uses all your experience.

Bias vs Variance

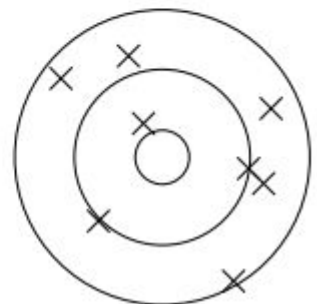
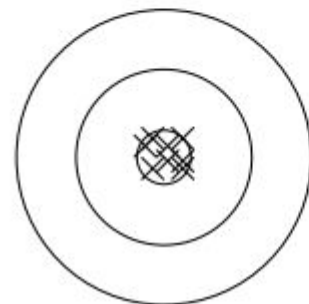
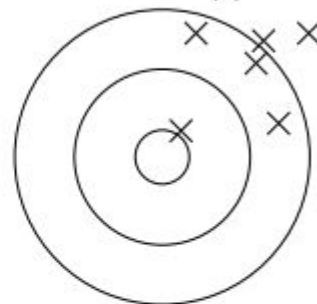
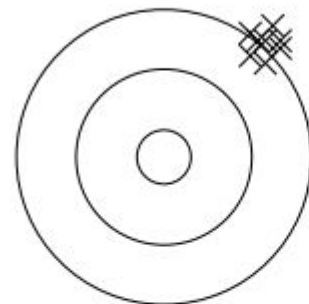


High
Bias

Low
Bias

Low
Variance

High
Variance



Bias–Variance Notes (First vs Every Visit)

- First–visit: described as unbiased, high variance
- Every–visit: bias decreases toward 0 with more trials
- Every–visit: described as lower variance, often faster convergence

Property	Bias	Variance	Typical Behavior
First-Visit MC	≈ 0 (unbiased)	Lower variance	Slower, smoother convergence
Every-Visit MC	≈ 0 (unbiased)	Higher variance	Faster but more fluctuation

The Goal

*We want the agent to learn the **best possible policy** (i.e., what action to take in every state) without knowing how the world works (no transition probabilities).*

This is called model-free control.

Quick Refresher: GPI (Generalized Policy Iteration)

Earlier (in model-based setups like Dynamic Programming), we used **GPI** — a two-step loop:

1. Policy Evaluation →

Find how good the current policy is (calculate values like $v_{\pi}(s)$).

2. Policy Improvement →

Use those values to make the policy slightly better (greedy update).

We repeat this until the policy becomes optimal.

Problem: No Model Now!

In Dynamic Programming, we could calculate everything because we *knew* how the world behaves (the transition dynamics $p(s', r|s, a)$).

But now, in the **model-free world**,
we don't know what happens when we take an action —
we can only **observe** outcomes by trying them (sampling).



Why State Values $v(s)$ Aren't Enough

To *improve* a policy, we need to pick the **best action** in each state.

That means we need to compare actions.

But $v(s)$ tells us only:

“How good is this state if I follow the policy from here?”

It doesn't tell us which **action** was better.

So instead of just $v(s)$, we now need **action values** $q(s, a)$:

$q(s, a)$ = expected return starting from state s , taking action a , then following the policy.

This tells us *how good each action* is —

so we can pick the best one using:

$$a^* = \arg \max_a q(s, a)$$

Simple Intuitive Example: Restaurant Choice

Imagine you're learning the best way to order food at your favorite restaurant.

- Each **state (s)** = situation (e.g., "Hungry evening").
- Each **action (a)** = what you order (pizza, pasta, salad).
- **Reward (r)** = how much you enjoyed it.
- **Policy (π)** = your current rule for ordering (maybe you always pick pasta).

Now, if you only know $v(s)$ = "average satisfaction on hungry evenings,"
you can't tell whether pizza or pasta was better.

You need $q(s, a)$ = "average satisfaction when ordering pizza *specifically* during hungry evenings."

That's why we move from **state values** to **action values**.



But There's Another Problem: Exploration

In MC, we learn by **sampling** (by actually playing and observing).

If the agent follows the same policy every time (say always choosing "pasta"), it will **never try** the other actions ("pizza" or "salad").

So the agent won't have enough data to estimate all $q(s, a)$.

Without knowing all $q(s, a)$, it can't know which one is best.



The Solution: Exploring Starts

To fix this, we force **exploration** —

we make sure that the agent **starts each episode from a random state-action pair**.

That way:

- Over many episodes, every state-action pair gets visited.
- We collect enough experience to estimate all $q(s, a)$.
- In the long run (with infinite samples), we can discover the true optimal policy.

It's like telling the restaurant critic:

"Every day, start with a random menu item and situation,
so over time you'll have tried *everything* at least once."

Step	Description	Intuition
1. Policy Evaluation	Estimate $q(s, a)$ by averaging sampled returns (from episodes)	Learn how good each action turned out to be
2. Policy Improvement	Make the policy greedy — choose the best action (highest q)	Choose what gives the best long-term reward
3. Exploration	Use exploring starts so every (s, a) gets tried	Try all possible menu items so you know what's best

Repeat → until policy stops changing →  optimal policy!

In model-free control, we don't know how the world works, so we learn by trying.

We estimate how good every action is in every situation (q -values) by sampling outcomes over many episodes, and we ensure we explore enough to cover all possibilities.

The Problem — “Exploring Starts” Is Unrealistic

Earlier we said:

“To make sure the agent learns about all actions, start each episode from a random state-action pair.”

That’s called **Exploring Starts** — and it ensures the agent eventually visits *every* (state, action) combination.

But in real life, that’s **not possible**:

- A **self-driving car** can’t start in random positions or speeds each episode.
- A **robot** can’t teleport to random places.
- It’s **wasteful** — visiting all (state, action) pairs infinitely is impossible.

So we need a **smarter way** to ensure the agent keeps exploring — even while learning a good policy.



The Solution — ϵ -Greedy Policy

The ϵ -greedy (epsilon-greedy) policy does exactly that.

It's a clever mix of exploration and exploitation.

When the agent is in a state s :

- With probability $1 - \epsilon$, it chooses the **best action** (the one with the highest q-value).
→ This is **exploitation** — use what it knows.
- With probability ϵ , it chooses a **random action**.
→ This is **exploration** — try something new.

So it keeps learning about new possibilities **without getting stuck** doing the same thing every time.

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|}, & \text{for } a = \arg \max_a Q(s, a) \\ \frac{\epsilon}{|A|}, & \text{otherwise} \end{cases}$$

Where:

- ϵ = exploration rate (e.g., 0.1 → 10% chance of exploring)
- $|A|$ = number of possible actions

Intuitive Example

Imagine you're a food critic trying to find the best dish at a restaurant.

- **Exploitation ($1 - \epsilon$):**

Most days, you order your current favorite dish (the one that tasted best so far).

- **Exploration (ϵ):**

Occasionally, you try something different — maybe a new chef's special — just to make sure there isn't something better.

That's ϵ -greedy learning — mostly stick to what's good, but sometimes experiment.



Making It Smarter Over Time

In the beginning, the agent doesn't know much — so it should explore more.
Later, when it's confident, it should explore less.

So we **reduce ϵ slowly** over time:

$$\epsilon \rightarrow 0$$

Meaning:

- Early $\rightarrow$ a lot of exploration
- Later $\rightarrow$ mostly greedy behavior (pure exploitation)

Eventually, it behaves greedily — but only after having explored everything enough.

This is what we call **"Greedy in the Limit with Infinite Exploration (GLIE)"**.



GLIE = The Practical Monte Carlo Control Strategy

Step	What Happens	Intuitive Meaning
1. Initialization	Start with random Q-values	The agent knows nothing yet
2. Run Episode	Follow ϵ -greedy policy	Try new things but mostly stick with good ones
3. Update $Q(s, a)$	Using sampled returns (Every-Visit MC)	Learn from experience
4. Improve Policy	Make policy greedy w.r.t. new Q	Choose best known actions more often
5. Reduce ϵ over time	Ensure GLIE	Gradually shift from exploring to exploiting

Repeat this cycle — and the agent eventually finds the **optimal policy**, even without a model.



Summary Intuition

ϵ -greedy is like being a curious learner:

- "You mostly go with what you know works best,"
- "But once in a while, you try something new — just in case it's better."

Over time, you become more confident and explore less.

That's how modern RL agents keep learning efficiently — no random starting points needed.

◆ Step 1: Initialization

Set up everything before learning begins.

1. $Q(s, a) = 0$
→ The agent starts with no knowledge about how good any (state, action) pair is.
("I don't yet know how good ordering pizza is in this restaurant.")
2. $N(s, a) = 0$
→ Count of how many times we've tried each action in each state.
("How many times have I ordered pizza when I was hungry?")
3. Policy π
→ Start with a random or highly exploratory policy.
("At the beginning, try random dishes to get a feel for things.")

◆ Step 2: Loop — Keep Learning from Episodes

We'll now repeat this process for many episodes.

Each episode = one full experience from start to end.

Think of it like one full game, one full meal, or one full driving trip.

◆ Step 5: Update Q-values (Every-Visit MC)

For each (state, action) in the episode — including repeated ones (Every-Visit):

1. Increment the visit count:

$$N(s, a) \leftarrow N(s, a) + 1$$

2. Update $Q(s, a)$ using an incremental average:

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} [G - Q(s, a)]$$

This means:

"Take the old estimate, compare it with the new sample (G), and move a small step toward it."

Each new episode slowly refines the Q -values — just like averaging experiences over time.

◆ Step 6: Reduce ϵ Over Time

$$\epsilon = \frac{1}{k}$$

As the number of episodes k increases:

- ϵ gets smaller
- The agent explores less
- It becomes more greedy (focused on best-known actions)

So early on, the agent tries lots of random things.

Later, it mostly chooses the best-known actions.

◆ Step 3: Sample an Episode

Follow the current policy (ϵ -greedy).

Record what happens:

$S_0, A_0, R_1, S_1, A_1, R_2, \dots, A_T$

That means: state $\rightarrow$ action $\rightarrow$ reward $\rightarrow$ next state $\rightarrow$ next action $\rightarrow \dots \rightarrow$ episode ends.

◆ Step 4: Work Backward to Compute Returns (G)

When the episode ends, we go **backward** from the last step.

For each time step t :

$$G \leftarrow \gamma \times G + R_{t+1}$$

This gives the total future return starting from that step.

◆ Step 7: Update Policy Using ϵ -Greedy

Now that $Q(s, a)$ is updated:

- Make the policy greedy — choose actions that maximize $Q(s, a)$.
- But still use ϵ -greedy to ensure continued exploration.

That way, the agent doesn't stop learning too early.

Initialize:

State-action values $Q(s, a) = 0$ for all $s \in S$ and $a \in A$.

Visit count $N(s) = 0$ for all $s \in S$.

Policy π with enough exploration e.g., random policy

Loop:

Sample episode (k) following policy π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, A_{T-1}, R_T, S_T$

$$G \leftarrow 0$$

Loop backward for each step of episode, $t = T - 1, T - 2, \dots, 1, 0$

$$G \leftarrow \gamma \cdot G + R_{t+1}$$

$$N(s, a) \leftarrow N(s, a) + 1$$

$$Q(s, a) \leftarrow Q(s, a) + 1 / N(s, a) * [G - Q(s, a)]$$

Reduce ϵ using $\epsilon = 1/k$

Update policy using ϵ -greedy using updated $Q(s, a)$



Intuitive Example: Food Critic Learning

Imagine a food critic (the agent) trying to find the best dish at a restaurant:

Step	Meaning	Example
Initialize	Start with no knowledge	No idea what dishes are good
Sample Episode	Try a few dishes per visit	Order soup, main, dessert, rate each
Compute Return	Add up total enjoyment	Dessert: 9/10, Main: 6/10, Soup: 7/10
Update $Q(s, a)$	Average your experiences	"Soup usually gives me 7.5/10 satisfaction."
ϵ -greedy	Keep some randomness	Occasionally try something new
Reduce ϵ	Explore less over time	Once confident, mostly order your favorite dish

Eventually, by balancing exploration and exploitation,
the critic discovers the **best dish** — the **optimal policy** 🍴

Numerical

- States: S_1, S_2 (both nonterminal), and then **terminal** after a second step.
- Actions in each state: **L** and **R** (so $|A| = 2$).
- Discount factor $\gamma = 1$ (no discounting — simplifies arithmetic).
- We will run **3 episodes** and update with **every-visit MC**.
- We use the GLIE rule $\epsilon = 1/k$ where k is the episode number (starting at 1).
- Initialization:
 - $Q(s, a) = 0$ for all s, a .
 - $N(s, a) = 0$ for all s, a .
 - Policy is ϵ -greedy based on current Q (initially random because Q 's equal).

We will **manually pick** the observed episodes (these are the samples the agent gets). The example episodes are small 2-step episodes so we can compute returns easily.

Notation reminder

- Episode k means we will set $\epsilon = 1/k$ for **policy used to generate that episode** (and also reduce after the episode).
- When updating $Q(s, a)$ we use incremental average:

$$N(s, a) \leftarrow N(s, a) + 1$$

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)} (G - Q(s, a))$$

(This is identical to $Q \leftarrow$ running average of samples.)

Episode 1 (k = 1)

- $\epsilon = 1/k = 1$. So policy is fully random: each action has probability $1/2$ in every state.
- Q table before episode: all zeros.

Sampled episode (observed by the agent):

State S_1 , action $\mathbb{L}$, reward $R_1 = 0 \rightarrow$ next state S_2

State S_2 , action $\mathbb{R}$, reward $R_2 = 1 \rightarrow$ terminal

Sequence: $S_1, L, 0, S_2, R, 1$, terminal.

We compute returns by going backward; $\gamma = 1$.

- Start $G \leftarrow 0$ at terminal.

Step t = 1 (state S_2 , action $\mathbb{R}$):

- $G \leftarrow \gamma \cdot G + R_2 = 1 \cdot 0 + 1 = 1$.
- Update counts and Q for (S_2, R) :
 - $N(S_2, R) \leftarrow 0 + 1 = 1$.
 - $Q(S_2, R) \leftarrow 0 + \frac{1}{1} (1 - 0) = 1.0$.

Step t = 0 (state S_1 , action $\mathbb{L}$):

- $G \leftarrow \gamma \cdot G + R_1 = 1 \cdot 1 + 0 = 1$.
- Update counts and Q for (S_1, L) :
 - $N(S_1, L) \leftarrow 0 + 1 = 1$.
 - $Q(S_1, L) \leftarrow 0 + \frac{1}{1} (1 - 0) = 1.0$.

Q and N after Episode 1:

- $Q(S_1, L) = 1.0$, $N(S_1, L) = 1$
- $Q(S_1, R) = 0.0$, $N(S_1, R) = 0$
- $Q(S_2, L) = 0.0$, $N(S_2, L) = 0$
- $Q(S_2, R) = 1.0$, $N(S_2, R) = 1$

Policy after Episode 1 (for next episode):

We compute ϵ for next episode as $\epsilon = 1/k = 1/1 = 1$. (So still fully random.)

But to see the ϵ -greedy probabilities according to the formula:

For a state s , for the action $a^* = \arg \max_a Q(s, a)$:

$$\pi(a^*|s) = 1 - \epsilon + \frac{\epsilon}{|A|}$$

All other actions get $\epsilon/|A|$.

Because $\epsilon = 1$:

- For each state, $\pi(\text{any action}|s) = 1/2$ (uniform). So policy remains random.

Episode 2 (k = 2)

- $\epsilon = 1/k = 1/2 = 0.5$.
- Current Q table is as above; we form ϵ -greedy policy from it:
 - In S_1 : $Q(L) = 1.0$, $Q(R) = 0.0 \rightarrow \arg \max = L$.
 - $\pi(L|S_1) = 1 - \epsilon + \epsilon/2 = 0.5 + 0.25 = 0.75$.
 - $\pi(R|S_1) = \epsilon/2 = 0.25$.
 - In S_2 : $Q(R) = 1.0$, $Q(L) = 0.0 \rightarrow \arg \max = R$.
 - $\pi(R|S_2) = 0.75$, $\pi(L|S_2) = 0.25$.

We sample an episode according to this policy. Suppose the sampled (observed) episode is:

State S_1 , action $\mathbb{R}$, reward $R_1 = 2 \rightarrow$ next S_2

State S_2 , action $\mathbb{L}$, reward $R_2 = 0 \rightarrow$ terminal

Sequence: $S_1, R, 2, S_2, L, 0$, terminal.

(We chose these sample actions to illustrate exploration: at S_1 the policy would choose $\mathbb{L}$ with $\frac{3}{4}$ but with prob 0.25 it explores and picks $\mathbb{R}$ — that happened here.)

Now update using every-visit MC (go backward):

- Start $G \leftarrow 0$.

Step t = 1 (state S_2 , action $\mathbb{L}$):

- $G \leftarrow 0 + 0 = 0$.
- Update (S_2, L) :
 - $N(S_2, L) \leftarrow 0 + 1 = 1$.
 - $Q(S_2, L) \leftarrow 0 + \frac{1}{1} (0 - 0) = 0.0$.

Step t = 0 (state S_1 , action $\mathbb{R}$):

- $G \leftarrow 0 + 2 = 2$.
- Update (S_1, R) :
 - $N(S_1, R) \leftarrow 0 + 1 = 1$.
 - $Q(S_1, R) \leftarrow 0 + \frac{1}{1} (2 - 0) = 2.0$.

Q and N after Episode 2 (cumulative):

- $Q(S_1, L) = 1.0$, $N(S_1, L) = 1$
- $Q(S_1, R) = 2.0$, $N(S_1, R) = 1$
- $Q(S_2, L) = 0.0$, $N(S_2, L) = 1$
- $Q(S_2, R) = 1.0$, $N(S_2, R) = 1$

Policy for next episode (k = 3):

- $\epsilon = 1/3 \approx 0.3333$.
- Compute ϵ -greedy probabilities:

At S_1 : $Q(L) = 1.0$, $Q(R) = 2.0$. $\arg \max$ is $\mathbb{R}$.

- $\pi(R|S_1) = 1 - \epsilon + \epsilon/2 = 1 - 0.3333 + 0.1667 = 0.8334$ (≈ 0.8333).
- $\pi(L|S_1) = \epsilon/2 = 0.1667$.

At S_2 : $Q(R) = 1.0$, $Q(L) = 0.0$. $\arg \max$ is $\mathbb{R}$.

- $\pi(R|S_2) = 0.8334$, $\pi(L|S_2) = 0.1667$.

(So by episode 3 the policy has become heavily biased toward action $\mathbb{R}$ in both states.)

Episode 3 (k = 3)

- $\epsilon = 1/3 \approx 0.3333$.
- Policy as above (mostly $\mathbb{R}$ in both states).

Suppose we sample the following episode (likely under this policy):

State S_1 , action $\mathbb{R}$, reward $R_1 = 1 \rightarrow S_2$

State S_2 , action $\mathbb{R}$, reward $R_2 = 2 \rightarrow$ terminal

Sequence: $S_1, R, 1, S_2, R, 2$, terminal.

Now update with every-visit MC (backward):

- Start $G \leftarrow 0$.

Step t = 1 (state S_2 , action $\mathbb{R}$):

- $G \leftarrow 0 + 2 = 2$.
- Update (S_2, R) :
 - Previous $N(S_2, R) = 1$, $Q(S_2, R) = 1.0$.
 - $N(S_2, R) \leftarrow 1 + 1 = 2$.
 - $Q(S_2, R) \leftarrow 1.0 + \frac{1}{2} (2 - 1.0) = 1.0 + 0.5 = 1.5$.

Step t = 0 (state S_1 , action $\mathbb{R}$):

- $G \leftarrow 2 + 1 = 3$ (since $\gamma = 1$)
- Update (S_1, R) :
 - Previous $N(S_1, R) = 1$, $Q(S_1, R) = 2.0$.
 - $N(S_1, R) \leftarrow 1 + 1 = 2$.
 - $Q(S_1, R) \leftarrow 2.0 + \frac{1}{2} (3 - 2.0) = 2.0 + 0.5 = 2.5$.

Final Q and N after Episode 3:

- $Q(S_1, L) = 1.0$, $N(S_1, L) = 1$
- $Q(S_1, R) = 2.5$, $N(S_1, R) = 2$
- $Q(S_2, L) = 0.0$, $N(S_2, L) = 1$
- $Q(S_2, R) = 1.5$, $N(S_2, R) = 2$

Interpretation & Intuition from the numbers

- **Every-visit updates:** each time we encountered a state-action pair we updated its Q — even repeated visits in an episode would be updated (we had two visits to (S_1, R) across episodes and two to (S_2, R)).
- **Counts** $N(s, a)$ show how many samples we used for each Q estimate. The running-average update used $1/N$ as the step size for each new sample, so newer estimates get blended in appropriately.
- **Policy evolution (ϵ decreasing):**
 - Episode 1: $\epsilon = 1.0 \rightarrow$ completely random policy (full exploration).
 - Episode 2: $\epsilon = 0.5 \rightarrow$ still exploration but biased toward actions with higher Q.
 - Episode 3: $\epsilon = 1/3 \rightarrow$ more greedy (mostly choosing the current best action).
- **Convergence behavior:** as k grows and the agent visits more (s,a) pairs, Q estimates stabilize (they converge to expected returns); and because $\epsilon \rightarrow 0$ as $k \rightarrow \infty$ (GLIE), the policy becomes greedy in the limit.

Off-Policy MC Control

In Reinforcement Learning, an agent learns a **policy** (a rule for choosing actions).

But there are **two possible ways to learn**:

Type	Idea
On-policy	Learn from the same policy you use to act
Off-policy	Learn about one policy while acting with another

In **GLIE MC control**, the agent:

1. Uses an **ϵ -greedy policy** to generate episodes.
2. Uses the **same policy** to update and improve itself.

So the **policy that generates experience = policy being optimized**.

Example:

Agent policy:

90% choose best action
10% explore

The agent **acts using this policy** and **learns to improve this same policy**.

That is **on-policy learning**.

Why On-Policy Can Be Limiting

Sometimes we want to:

- **Explore a lot** while collecting data
- But **learn a very greedy optimal policy**

Example:

A **self-driving car** learning:

- While learning → try many actions (explore more)
- Final policy → always choose the safest best action

But in **on-policy**, exploration and learning are tied together.

So we need a better idea.

Off-Policy Learning (The Key Idea)

Off-policy separates **two** policies.

Policy

Purpose

Behavior Policy

Used to **generate data / explore**

Target Policy

The policy we want to **learn / optimize**

So the agent:

Act using Policy A
Learn about Policy B

This is **off-policy learning**.

Intuition: The behavior policy generates data, but the target policy is the one we want to learn.

Behavior policy (exploring)

You behave like this:

Try random dishes often
Try new items
Experiment

This helps you **collect lots of information**.

Target policy (optimal)

But the policy you want to learn is:

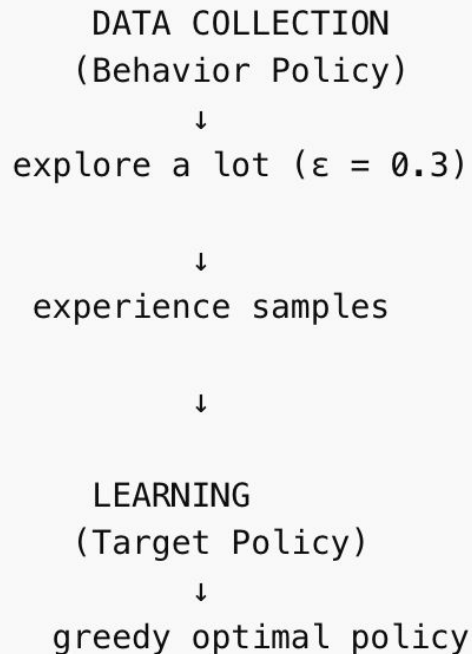
Always order the best dish

So:

Action	Policy used
Collect experience	exploratory policy
Learn best action	greedy optimal policy

This is **off-policy learning**.

Think of two brains:



Why Off-Policy is Powerful

Off-policy allows:

1 More exploration

Behavior policy can explore a lot.

2 Faster learning

Target policy can stay greedy.

3 Learning from other agents

You can learn from **someone else's data**.

Example:

- Learn driving from **recorded driving logs**
- Learn chess from **grandmaster games**

The behavior policy is **not yours**, but you still learn the optimal policy.

Quick Comparison

Feature	On-Policy	Off-Policy
Data generated by	Same policy	Different policy
Exploration	Limited by policy	Can be very exploratory
Policy learned	Same one	Target policy
Example	GLIE MC control	Off-policy MC, Q-learning

One-Line Intuition

On-policy

Learn from what *you actually do*.

Off-policy

Learn the best strategy *even while behaving differently*.

✓ Simplest mental picture

On-policy:

Act = Learn

Off-policy:

Act with Policy A

Learn Policy B

Initialize:

State-action values $Q(s, a) = 0$ for all $s \in S$ and $a \in A$.

Visit count $N(s) = 0$ for all $s \in S$.

Policy $\pi = \operatorname{argmax}_a Q(S, a)$

Loop:

$b \leftarrow$ a “behavior policy” with enough exploration

Sample episode (k) following policy b : $S_0, A_0, R_1, S_1, A_1, \dots, A_{T-1}, R_T, S_T$

$G \leftarrow 0$

Loop backward for each step of episode, $t = T - 1, T - 2, \dots, 1, 0$

$$G \leftarrow \gamma \cdot G + R_{t+1}$$

$$N(s, a) \leftarrow N(s, a) + 1$$

$$Q(s, a) \leftarrow Q(s, a) + 1 / N(s, a) * [G - Q(s, a)]$$

$$\pi = \operatorname{argmax}_a Q(S, a)$$

1. Initialization (Starting everything)

From the image:

$Q(s,a) = 0$ for all states and actions
 $N(s,a) = 0$
 $\pi = \operatorname{argmax}_a Q(s,a)$

Meaning

$Q(s,a)$

This stores:

"How good is action a in state s ?"

Initially we **know nothing**, so we set everything to 0.

Example table:

State	Action	$Q(s,a)$
S1	A1	0
S1	A2	0
S2	A1	0
S2	A2	0

$N(s,a)$

Counts:

"How many times did we try action a in state s ?"

Initially all 0.

Policy π

$\pi = \operatorname{argmax}_a Q(s,a)$

Meaning:

Choose the action with the **highest Q value**.

2. Behavior Policy b

$b \leftarrow a$ "behavior policy" with enough exploration

This means:

We use a **random** or **ϵ -greedy** policy to explore.

Example:

State S1

Action	Probability
A1	0.5
A2	0.5

Why?

Because if we always choose the best action, we **may miss better ones**.

So behavior policy **explores more**.

3. Generate an Episode

Sample episode following policy b
 $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$

This means we **play one full game**.

Example episode:

Step	State	Action	Reward
0	S1	A2	3
1	S2	A1	5
2	Terminal	—	—

So the episode is:

$S_1 \rightarrow A_2 \rightarrow \text{reward } 3 \rightarrow S_2 \rightarrow A_1 \rightarrow \text{reward } 5 \rightarrow \text{End}$

5. Go Backwards Through Episode

for $t = T-1, T-2, \dots, 0$

Monte Carlo methods **process from the end backward**.

Why?

Because when you're at the end you already know the **future rewards**.

6. Update the Return

$G \leftarrow \gamma G + R(t+1)$

Meaning:

$\text{Return} = \text{discount} \times \text{future return} + \text{immediate reward}$

Example ($\gamma = 1$ for simplicity):

Last step reward = 5

$G = 0$
 $G = 1 \cdot 0 + 5$
 $G = 5$

Now go to previous step:

Reward = 3

$G = 1 \cdot 5 + 3$
 $G = 8$

So:

Step	Return
(S2,A1)	5
(S1,A2)	8

8. Update Q value

Formula:

$$Q(s,a) = Q(s,a) + (1/N(s,a)) * (G - Q(s,a))$$

This is just a **running average**.

Think of it as:

New Q value = average of all returns seen so far.

Example

For (S2,A1):

Q = 0
N = 1
G = 5

Update:

$$Q = 0 + (1/1) * (5 - 0)$$
$$Q = 5$$

Next pair:

For (S1,A2)

Q = 0
N = 1
G = 8

Update:

$$Q = 8$$

Now table becomes:

State	Action	Q
S1	A1	0
S1	A2	8
S2	A1	5
S2	A2	0

9. Improve the Policy

$$\pi = \operatorname{argmax}_a Q(s,a)$$

Meaning:

Choose the **best action based on Q values**.

Example:

State S1

Action	Q
A1	0
A2	8

Best action = **A2**

So policy becomes:

$$\pi(S1) = A2$$

10. Repeat Many Episodes

We repeat:

Generate episode → Update Q → Improve policy

Eventually:

- Q values become accurate
- Policy becomes **optimal**

The Whole Idea in One Sentence

This algorithm:

Explores using a random behavior policy, learns Q values using Monte Carlo returns, and gradually improves the target policy to become optimal.

Core Problem with standard Off Policy Algorithm

In **Off-Policy Learning**, we have **two** policies:

Behavior Policy b

The policy we **use to generate episodes**.

Example:

Action	Probability
Left	0.5
Right	0.5

It is **random / exploratory**.

Target Policy π

The policy we **want to evaluate or learn**.

Example:

Action	Probability
Left	1
Right	0

It is usually **greedy (best action)**.



The Big Problem

We want to know:

What reward would we get if we followed π ?

But the data we collected was from **b**.

So the experience is **biased**.

Example:

Episode generated by **b**:

S1 $\rightarrow$ Right $\rightarrow$ reward 5

But target policy π might be:

S1 $\rightarrow$ Left

So this episode **doesn't match** π .

Idea of Importance Sampling

Importance Sampling says:

If the behavior policy produced something that the target policy also likes, **give it high weight**.

If the behavior policy produced something the target policy would rarely do, **give it low weight**.

Weight Formula

The importance weight is:

$$W = \frac{\pi(A|S)}{b(A|S)}$$

Meaning:

weight = $\frac{\text{probability under target policy}}{\text{probability under behavior policy}}$

Simple Example

Suppose:

Behavior policy b :

Action	Probability
Left	0.5
Right	0.5

Target policy π :

Action	Probability
Left	1
Right	0

Case 1: Behavior chose LEFT

Episode step:

$S_1 \rightarrow \text{Left}$

Weight:

$$\begin{aligned} W &= \pi(\text{Left} | S_1) / b(\text{Left} | S_1) \\ W &= 1 / 0.5 \\ W &= 2 \end{aligned}$$

Meaning:

This action **matches the target policy**, so we **increase its importance**.

Case 2: Behavior chose RIGHT

Episode step:

$S_1 \rightarrow \text{Right}$

Weight:

$$\begin{aligned} W &= 0 / 0.5 \\ W &= 0 \end{aligned}$$

Meaning:

Target policy would **never do this**, so **ignore this sample**.

Intuition

Imagine you're studying **how a professional chess player plays**.

But the games you recorded were played by **beginners**.

So you do this:

1. Watch beginner games.
2. Check how similar their moves are to the professional strategy.
3. If the move matches the professional move → **give it high importance**.
4. If the move is bad → **ignore it**.

That weighting process is **Importance Sampling**.

Full Algorithm With Importance Sampling

```
Initialize  $Q(s,a)$   
Initialize  $C(s,a) = 0$   
 $\pi(s) = \operatorname{argmax}_a Q(s,a)$   
  
Loop forever:  
  
Generate episode using behavior policy  $b$   
  
 $G = 0$   
 $W = 1$   
  
for  $t = T-1$  to  $0$ :  
  
     $G = \gamma G + R(t+1)$   
  
     $C(S_t, A_t) += W$   
  
     $Q(S_t, A_t) += (W/C(S_t, A_t)) * (G - Q(S_t, A_t))$   
  
     $\pi(S_t) = \operatorname{argmax}_a Q(S_t, a)$   
  
    if  $A_t \neq \pi(S_t)$ :  
        break  
  
     $W = W * \pi(A_t|S_t) / b(A_t|S_t)$ 
```

Numerical

1. Problem Setup

Assume we have one state S and two actions:

Action	Meaning
A	Good action
B	Bad action

Target Policy π

The policy we want to learn.

Action	Probability
A	1
B	0

So the target policy **always chooses A**.

Behavior Policy b

The policy used to **generate episodes**.

Action	Probability
A	0.5
B	0.5

So behavior policy is **random**.

Discount Factor

$$\gamma = 1$$

(for simplicity)

Initial Values

$Q(S,A)$	$Q(S,B)$
0	0
$C(S,A)$	$C(S,B)$
0	0

Here:

- $Q(s,a)$ = estimated value
- $C(s,a)$ = cumulative importance weight

2. Episode Generated Using Behavior Policy

Suppose the behavior policy produced this episode:

$$S \rightarrow A \rightarrow \text{reward} = 4 \rightarrow \text{terminal}$$

So the episode is:

t	State	Action	F
0	S	A	4

3. Initialize Variables

$$\begin{aligned} G &= 0 \\ W &= 1 \end{aligned}$$

Where:

- G = return
- W = importance weight

4. Start Backward Loop

There is only **one step**.

$$t = 0$$

Step 1 — Compute Return

Formula:

$$G = \gamma G + R_{t+1}$$

Substitute values:

$$\begin{aligned} G &= 1 * 0 + 4 \\ G &= 4 \end{aligned}$$

Step 2 — Update C(s,a)

Formula:

$$C(s,a) = C(s,a) + W$$

Substitute:

$$\begin{aligned} C(S,A) &= 0 + 1 \\ C(S,A) &= 1 \end{aligned}$$

Step 3 — Update Q(s,a)

Formula:

$$Q(s,a) = Q(s,a) + \frac{W}{C(s,a)} (G - Q(s,a))$$

Substitute:

$$\begin{aligned} Q(S,A) &= 0 + (1/1) * (4 - 0) \\ Q(S,A) &= 4 \end{aligned}$$

Updated Table

State	Action	Q
S	A	4
S	B	0

Step 4 — Update Target Policy

Policy becomes:

$$\pi(S) = \operatorname{argmax} Q(s,a)$$

So:

$$\pi(S) = A$$

Step 5 — Check if Action Matches Target Policy

We check:

$$\text{Was action A equal to } \pi(S)?$$

Yes.

So we continue.

Step 6 — Update Importance Weight

Formula:

$$W = W \times \frac{\pi(A|S)}{b(A|S)}$$

Substitute values:

Target policy probability:

$$\pi(A|S) = 1$$

Behavior policy probability:

$$b(A|S) = 0.5$$

So:

$$\begin{aligned} W &= 1 * (1 / 0.5) \\ W &= 2 \end{aligned}$$

Meaning:

This episode becomes **twice as important** because the behavior policy rarely picks the target action.

5. Now Let's See Another Episode

Suppose behavior policy generates:

$S \rightarrow B \rightarrow \text{reward} = 3$

Episode:

State	Action	Reward
S	B	3

Step 1 — Return

$G = 3$

Step 2 — Update C

$C(S, B) = 0 + 1$
 $C(S, B) = 1$

Step 3 — Update Q

$Q(S, B) = 0 + (1/1) * (3 - 0)$
 $Q(S, B) = 3$

Step 4 — Policy Update

Action	Q
A	4
B	3

So:

$\pi(S) = A$

Step 5 — Check Action

Behavior action was B.

Target policy action is A.

So:

$B \neq A$

Therefore:

Break the loop

And importance weight becomes 0.

Meaning:

This episode **does not help evaluate the target policy**.

7. Key Intuition

Importance Sampling is doing this:

Episode	Behavior Action	Target Action	Weight
1	A	A	High (2)
2	B	A	0

So learning mainly uses **episodes that match the target policy**.

Importance sampling **corrects the bias** caused by using a different policy to generate data.

6. Final Learned Values

Action	Q
A	4
B	3

Target policy chooses:

$\pi(S) = A$

